



SHL Conversational Recommender Assignment

Here's the **exact breakdown** of what the PDFs say we need to build and use:

- **Endpoints:** A FastAPI service with **two endpoints**:
 - `GET /health` – a simple readiness check (should return HTTP 200 OK when up).
 - `POST /chat` – the main chat endpoint. It takes a JSON body `{"messages": [...]}` (full conversation history, stateless) and returns a JSON reply.
- **POST /chat schema (non-negotiable!):** The request must be

```
{
  "messages": [
    {
      "role": "user" or "assistant",
      "content": "<text>"
    },
    ...
  ]
}
```

and the response must be a JSON object with keys:

```
{
  "reply": "<assistant reply text>",
  "recommendations": [
    {
      "name": "<assessment name>",
      "url": "<SHL link>",
      "test_type": "<type>"
    },
    ...
  ],
  "end_of_conversation": true or false
}
```

In other words, every POST returns the next assistant message (`reply`), plus a **structured shortlist** of recommended SHL assessments when appropriate. If the conversation is finished, set `"end_of_conversation": true`; otherwise `false`. **Do not deviate** from this schema or our evaluator will break **[6†L9-L11] [27†L1-L4]** .

- **Behavior of /chat:** The agent should guide a hiring manager from vague intent to a precise set of SHL assessments. That means:
 - Ask clarifying questions (e.g. seniority, tech stack) if needed.
 - Accept refinements or corrections (e.g. user changes requirements mid-conversation).
 - Never recommend anything **outside the provided SHL catalog**. (Catalog items only.)
 - Compare assessments when asked.
 - Handle personalities and facts from the provided traces (see below).

- Examples in the sample conversations show this flow (e.g. page 4 of the assignment PDF with a Java example) [27†L1-L4] .
- **Data Provided:**
 - **SHL Product Catalog:** A JSON catalog of SHL assessments (377 items) was provided (see `shl_product_catalog.pdf`). Use this as your source of all `name` / `url` / `test_type` data. (For each recommendation, pick an item from this list.)
 - **Conversation Traces:** 10 provided “public conversation traces” (C1.md through C10.md). Each trace is a persona with *facts* plus the correct shortlist of assessments (ground truth). These are your development set. Treat them as examples to guide your logic. (In evaluation, the same traces will be used to score your agent’s Recall@K and behavior.)
 - **Evaluation Constraints (must-haves):** According to the assignment, scoring has **three parts** [28†L1-L8] :
 - **Hard requirements (mandatory):**
 - **JSON Schema compliance:** Every response must follow the exact schema above (reply, recommendations, end_of_conversation).
 - **Catalog-only:** Only use items from the SHL catalog in `recommendations` . No made-up items or off-catalog suggestions [28†L1-L8] .
 - **Turn limit:** Cap each conversation at **8 total turns** (including both user and assistant). After 8 turns or when done, set `end_of_conversation: true` [28†L1-L8] [29†L1-L4] .
 - **Recommendation quality:**
 - **Recall@10:** We’ll evaluate the final recommendations against the trace’s ground truth. Aim for high Recall@10 (fraction of relevant assessments in your top 10 list) [28†L1-L8] .
 - **Behavior probes (qualitative checks):** The agent should “pass” various behavior tests, e.g.:
 - Refuse clearly off-topic requests politely.
 - Do **not** recommend on turn 1 if the query is still too vague.
 - Honor user edits (e.g. if user says “actually prefer mid-level”).
 - **No hallucinations:** Don’t invent fake facts or products. Only use what’s given (facts/persona) and catalog data [28†L1-L8] .
 - Overall: logical, helpful, and rooted in the provided data (per page 5 example).

In short: build a FastAPI chat service with `/health` and `/chat` exactly as specified. Use the SHL catalog JSON and the sample traces. Keep conversations grounded, within 8 turns, always using real catalog items, and follow the JSON format to the letter.

API Endpoints and Schema

- **GET /health:** Return a simple 200 OK (e.g. `{"status": "ok"}`) to signal readiness.
- **POST /chat:** Expect JSON `{"messages": [...]}` (full history). Return:

```
{
  "reply": "<assistant text>",
  "recommendations": [
```

```

    {"name": "...", "url": "...", "test_type": "..."},
    ...
  ],
  "end_of_conversation": true_or_false
}

```

Example: In a sample (page 4 of the assignment PDF) the user asks about seniority, and the response JSON is:

```

{
  "reply": "Got it. Here are 5 assessments that fit a mid-level Java dev with stakeholder needs.",
  "recommendations": [
    {"name": "Java 8 (New)", "url": "https://www.shl.com/...", "test_type": "K"},
    {"name": "OPQ32r", "url": "https://www.shl.com/...", "test_type": "P"}
  ],
  "end_of_conversation": false
}

```

(This matches the **required schema** [27†L1-L4] .)

Data and Conversation Traces

- **SHL Catalog:** The `shl_product_catalog.pdf` is a pretty-printed JSON list of SHL assessment products. Each entry has `"entity_id"`, `"name"`, `"url"`, `"test_type"`, etc. We'll use this as our database of assessments. For each recommendation, return the `name`, `url`, and `test_type` exactly as given.
- **Conversation Traces:** Files C1–C10 contain example dialogs. Each starts with a persona's facts and shows a turn-by-turn conversation concluding with a correct recommendation table. For instance, C1's final answer lists several product links and then `end_of_conversation: true`. Use these to guide behavior and to test your logic. (They form the "development set" and ground truth for Recall@K [28†L1-L8] .)

Evaluation Requirements

Be aware that the evaluator will enforce these constraints:

- **Strict JSON format:** Do **not** add extra fields. The bot's responses must be valid JSON matching the schema above (reply + recommendations + end_of_conversation) [28†L1-L8] .
- **Catalog-only recommendations:** Only items from the provided catalog may be recommended. Check by name or ID. (The assignment explicitly says "Items from catalog only in recommendations" [28†L1-L8] .)
- **Turn cap = 8:** After 8 turns total, you must end the conversation (`"end_of_conversation": true`). Even if still more discussion possible, the turn limit is hard.

- **Recall@10:** You should return up to 10 recommendations. The score will be the mean Recall@10 over test traces. (So focus on ranking relevance within your top-10 list.)
- **No hallucinations:** Don't invent any details or products. Stick to the user's facts (from traces) and the catalog data. For example, if the user never mentioned "Python", your bot shouldn't start talking about Python tests.
- **Behavior probes:** The evaluator will check small edge cases, e.g. refusing to work off-topic, not recommending on turn 1 when query is vague, honoring user corrections, etc. Follow the examples given (see page 5 for typical probe descriptions [\[29†L3-L9\]](#)).

Replit Implementation Prompt

When writing your code prompt for Replit (or any LLM assistant), make it **concise and actionable**. For example:

```
Develop a FastAPI backend for the SHL Conversational Recommender. Implement two endpoints:
- GET /health: returns HTTP 200 OK (e.g. {"status": "ok"}).
- POST /chat: accepts JSON {"messages": [...]} (chat history) and returns JSON:
{
  "reply": "...",
  "recommendations": [ {name, url, test_type}, ... ],
  "end_of_conversation": true/false
}
Use the provided SHL catalog JSON as the database for assessments (377 items).
Always pick recommendations from this catalog. Follow the JSON schema exactly.
Use the sample conversation traces to handle clarifications and context. Enforce
a maximum of 8 turns per session. Only recommend relevant catalog items (up to
10). Ensure the agent asks follow-up questions when needed, accepts edits, and
never hallucinates.
```

This prompt tells Replit **exactly what to build** (endpoints, data, constraints) in a clear, bullet-point style. It's succinct but hits every requirement from the assignment PDF.